

# Documentation technique - Vite & Gourmand

Projet ECF - TP Développeur Web et Web Mobile (Studi)

Auteur : Florian Marques

Application en ligne : <https://vite-et-gourmand-bxiy.onrender.com>

Dépôt : <https://github.com/Xirdrak/vite-et-gourmand>

---

## Sommaire

---

1. Réflexions initiales technologiques
  2. Configuration de l'environnement de travail
  3. Modèle de données (MCD / diagramme de classes)
  4. Diagramme de cas d'utilisation
  5. Diagramme de séquence (commande d'un menu)
  6. Documentation du déploiement
- 

## 1. Réflexions initiales technologiques

---

### 1.1 Le besoin

Vite & Gourmand est un traiteur événementiel bordelais qui envoie aujourd'hui ses menus par mail à ses habitués. L'objectif du projet est de remplacer ce fonctionnement par une application web qui doit permettre :

- de présenter les menus au public (visiteurs non connectés) ;
- de prendre des commandes en ligne (utilisateurs connectés) ;
- de gérer les menus, les commandes et les avis (employés et administrateur) ;
- de suivre des statistiques de commandes et de chiffre d'affaires (administrateur).

L'application doit gérer quatre rôles (visiteur, utilisateur, employé, administrateur), un workflow de commande à plusieurs états, des mails transactionnels, et un tableau de bord statistique alimenté par une base NoSQL.

### 1.2 La stack retenue et ses justifications

J'ai retenu des technologies vues en formation, que je pouvais donc prendre en main sans les découvrir en même temps que le projet. Pour une application que je dois présenter à l'oral, il valait mieux m'appuyer sur des outils que je connais plutôt que sur une stack plus récente que je maîtriserais mal.

| Couche   | Technologie         | Justification                                                                                                                                                                |
|----------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Back-end | PHP 8.5 / Symfony 8 | Framework MVC complet (routing, sécurité, ORM, formulaires, mailer). Imposé par le cadre de la formation, et adapté à une application métier avec authentification et rôles. |

| Couche            | Technologie                    | Justification                                                                                                                                                      |
|-------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Templates         | Twig                           | Moteur de template intégré à Symfony, séparation propre vue / logique, échappement automatique du HTML (protection XSS par défaut).                                |
| Front-end         | HTML / CSS custom / JS vanilla | Pas de framework JS : le périmètre fonctionnel ne le justifie pas. Les interactions dynamiques (filtres menus, récapitulatif de commande) sont faites en JS natif. |
| ORM               | Doctrine                       | Mapping objet-relationnel standard de Symfony, requêtes sécurisées (requêtes paramétrées contre l'injection SQL).                                                  |
| BDD relationnelle | MySQL 9                        | Données transactionnelles (utilisateurs, commandes, menus). Intégrité référentielle par clés étrangères.                                                           |
| BDD NoSQL         | MongoDB (Atlas)                | Imposée par le sujet pour les statistiques admin. Alimentée par une synchronisation depuis MySQL.                                                                  |
| Hébergement       | Render (Docker)                | Free tier web service sans carte bancaire (voir 1.3).                                                                                                              |
| BDD managées      | Aiven (MySQL) + MongoDB Atlas  | Bases hébergées à l'extérieur de l'application : le conteneur ne contient que le code, pas les données, donc il se redéploie sans risque pour la base.             |
| Mails             | Mailtrap (SMTP de test)        | Capture les mails transactionnels sans envoi réel, suffisant pour une démo ECF.                                                                                    |

## Pourquoi CSS custom plutôt que Bootstrap

Bootstrap est suggéré par le sujet mais non obligatoire. J'avais déjà fait une maquette Figma complète pour ce projet (palette, typographie, composants). Partir de Bootstrap m'aurait obligé à surcharger la quasi-totalité de ses variables pour coller à la maquette. Un CSS custom organisé autour de variables CSS est plus cohérent avec le design et plus simple à maintenir. J'ai donc retiré Bootstrap de `base.html.twig` et je gère le responsive avec des media queries.

## Pourquoi deux bases de données

Le sujet impose une base NoSQL pour les statistiques de l'administrateur. Je n'avais jamais utilisé MongoDB avant ce projet, donc j'ai pris le temps de comprendre comment m'en servir sans casser ce qui marchait déjà. J'ai gardé MySQL pour toutes les données du site (utilisateurs, menus, commandes) et je me sers de MongoDB seulement pour les stats, comme demandé. Une commande est donc d'abord écrite dans MySQL, au moment où le client valide.

La vraie question, c'est comment les stats se retrouvent dans MongoDB alors que les commandes sont dans MySQL. Pour ça, j'ai écrit une commande Symfony `app:stats:sync` (avec un bouton "Resynchroniser" côté admin) qui va lire les commandes dans MySQL et les recopier une par une dans une collection MongoDB. Le tableau de bord ne lit jamais MySQL pour les stats, il lit uniquement cette collection MongoDB.

Pour calculer les chiffres, MongoDB utilise ce qu'on appelle un pipeline d'agrégation. Concrètement, c'est une suite d'étapes que la base applique aux documents les uns après les autres. Dans mon cas, je regroupe les commandes par menu (étape `$group`), et dans le même mouvement je compte le nombre

de commandes et j'additionne le chiffre d'affaires, puis je trie le résultat (étape `$sort`). C'est l'équivalent d'un `GROUP BY` en SQL, juste écrit sous forme d'étapes qui s'enchaînent.

J'ai choisi de recopier les données plutôt que d'écrire chaque commande dans les deux bases en même temps, pour deux raisons. D'abord, si MongoDB est indisponible, ça n'empêche pas de commander, la commande s'enregistre quand même dans MySQL. Ensuite, écrire dans les deux bases au moment de la commande m'aurait obligé à retoucher le code de commande déjà terminé et testé, alors que la synchro est un ajout à part qui ne touche à rien d'existant.

Le compromis que j'assume, c'est que les stats ne sont pas en temps réel. Il faut relancer la synchro pour qu'une nouvelle commande apparaisse dans les chiffres. Pour ne pas créer de doublons quand je relance, la recopie utilise un `replaceOne` avec l'option `upsert` : chaque commande est repérée par son `numero_commande`, donc si elle est déjà dans MongoDB elle est remplacée, sinon elle est créée. Je peux relancer la synchro autant de fois que je veux, le résultat reste le même.

### 1.3 Alternatives envisagées et arbitrages

- **Hébergement : Fly.io puis Render.** Fly.io était le choix initial (cité par la formation), mais il exige désormais une carte bancaire dès la création d'une application. Koyeb aussi. J'ai donc basculé sur Render, dont le free tier web service Docker ne demande pas de CB. Ça m'a coûté peu d'efforts : l'application ne porte que le conteneur applicatif (les bases sont managées à l'extérieur), il a suffi de remplacer `fly.toml` par `render.yaml` et de réutiliser le Dockerfile tel quel. Limite que j'assume : sur le free tier, la machine se met en veille après 15 minutes d'inactivité (cold start au réveil, environ 50 secondes), ce qui est acceptable pour une démo et que je mentionnerai en soutenance.
- **Accès à MongoDB : Atlas Data API puis driver natif.** Le plan initial visait l'Atlas Data API (accès HTTP REST à MongoDB, qui aurait évité d'installer une extension PHP). MongoDB a arrêté cette API le 30/09/2025. Je suis donc passé par le driver natif `ext-mongodb` (build 2.3.3 pour PHP 8.5) plus la librairie `mongodb/mongodb`. Leçon retenue : vérifier la disponibilité des extensions natives pour sa version exacte de PHP avant de s'engager sur une techno.
- **MongoDB local vs Atlas.** J'ai préféré Atlas (cluster gratuit M0) à une installation locale, parce qu'une seule connection string fonctionne en dev comme en prod, sans migration de données entre environnements ni configuration réseau supplémentaire au déploiement.

---

## 2. Configuration de l'environnement de travail

---

Le développement s'est fait sous Windows 11. La procédure complète d'installation en local figure dans le `README.md` du dépôt. Les versions réelles utilisées sont les suivantes.

| Outil    | Version | Rôle                            |
|----------|---------|---------------------------------|
| PHP      | 8.5.6   | Langage back-end                |
| Composer | 2.9.8   | Gestionnaire de dépendances PHP |

| Outil          | Version                                                     | Rôle                                 |
|----------------|-------------------------------------------------------------|--------------------------------------|
| Symfony CLI    | 5.17.1                                                      | Serveur de dev local, outils Symfony |
| MySQL          | 9.7.0 (Community)                                           | Base relationnelle locale            |
| MongoDB        | Atlas M0 (cloud) + ext-mongodb 2.3.3 / mongodb-lib 2.3.0    | Base NoSQL                           |
| Extensions PHP | pdo_mysql, intl, mbstring, openssl, xml, curl, zip, mongodb | Extensions requises                  |

## Mise en place rapide (résumé du README)

```
git clone <url-du-repo> vite-et-gourmand
cd vite-et-gourmand/app
composer install
cp .env .env.local          # renseigner DATABASE_URL et MONGODB_URI
mysql -u <user> -p <bdd> < ../schema.sql
mysql -u <user> -p <bdd> < ../seed.sql
symfony server:start
```

L'application est alors accessible sur `https://localhost:8000`.

## Organisation du dépôt

```
/
├─ app/           Application Symfony (code, config, templates, assets)
├─ schema.sql     Création des tables MySQL (SQL brut, exigé par le sujet)
├─ seed.sql       Données de test (SQL brut)
├─ Dockerfile     Image de production (PHP-fpm + nginx + supervisor)
├─ render.yaml    Description du service Render (infra as code)
├─ livrables/     Documents PDF du projet
└─ README.md
```

Les fichiers SQL bruts sont à la racine, comme exigé par le sujet (pas de migration ni de fixture Doctrine en remplacement).

## Gestion des secrets

Les secrets ne sont jamais versionnés. En local ils vivent dans `app/.env.local` (gitignore), en production ils sont injectés par le dashboard Render (déclarés `sync: false` dans `render.yaml`). Le fichier `.env` versionné ne contient que des placeholders, pour que la liste des variables attendues soit visible. Cette séparation suit le facteur III "Config" de la méthodologie Twelve-Factor App.

### 3. Modèle de données (MCD / diagramme de classes)

#### 3.1 Démarche : analyse critique du MCD fourni

Le sujet fournit un MCD en annexe qui sert de base de référence obligatoire. Mais ce MCD a plusieurs incohérences et quelques manques par rapport à l'énoncé fonctionnel. J'ai donc choisi de partir du MCD fourni, puis de documenter et justifier chaque écart, plutôt que de le recopier tel quel ou de le réinventer dans mon coin.

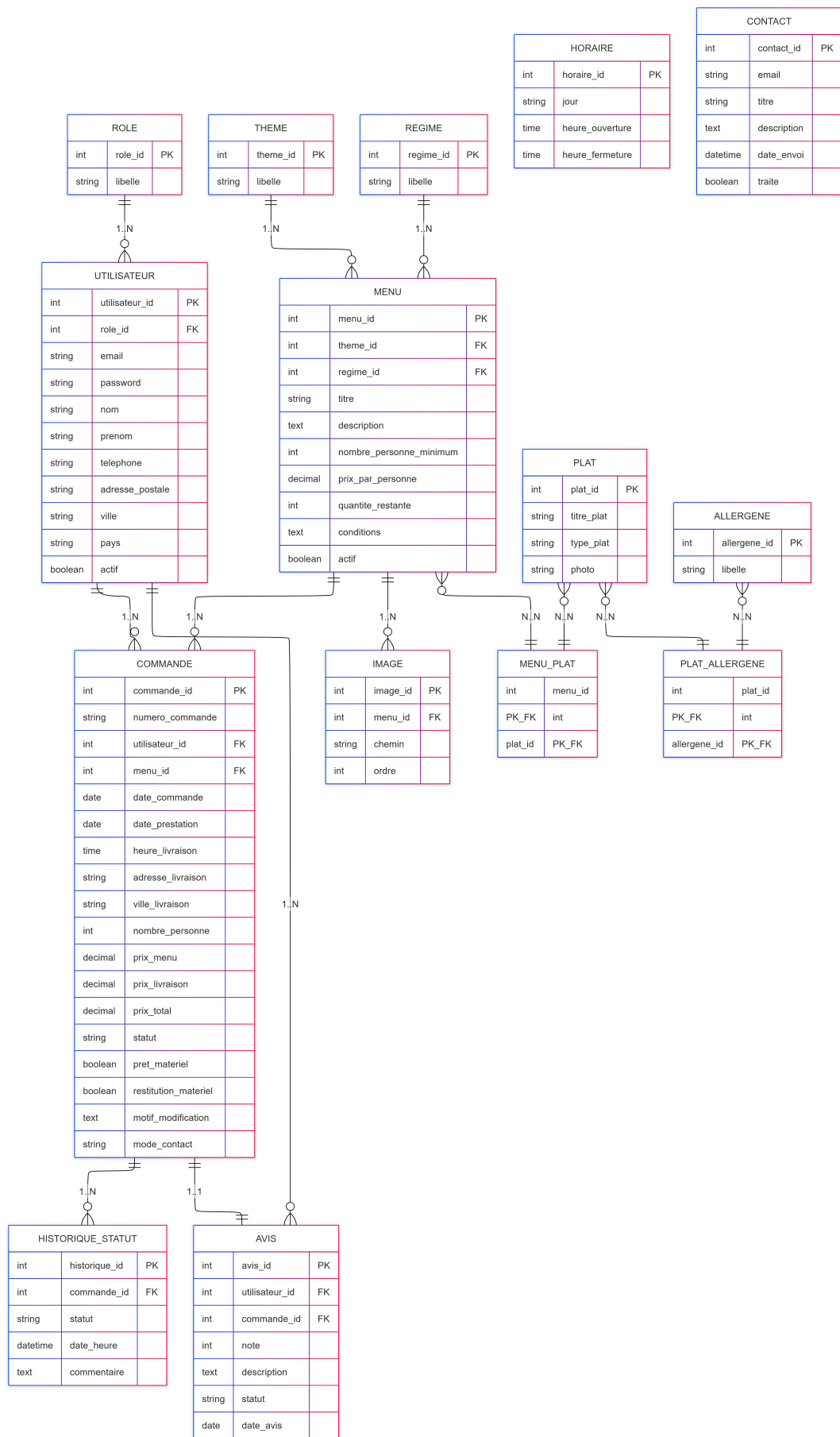
J'ai identifié et corrigé douze écarts :

| #  | Écart constaté dans le MCD                                                                    | Correction apportée                                                                                           |
|----|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 1  | Champ <code>nom</code> manquant dans <code>utilisateur</code> (l'énoncé demande nom + prénom) | Ajout de la colonne <code>nom</code> <code>VARCHAR(100)</code>                                                |
| 2  | <code>password</code> en <code>VARCHAR(50)</code> , trop court pour un hash                   | Passage à <code>VARCHAR(255)</code>                                                                           |
| 3  | Colonne <code>regime</code> en doublon dans <code>menu</code> (existe aussi en table)         | Suppression de la colonne, usage unique de la FK <code>regime_id</code>                                       |
| 4  | Galerie d'images absente, <code>photo</code> en BLOB sur <code>plat</code>                    | Table <code>image</code> dédiée + chemins de fichiers ( <code>VARCHAR</code> ) au lieu de BLOB                |
| 5  | Pas de suivi des changements de statut                                                        | Table <code>historique_statut</code> (statut + horodatage par commande)                                       |
| 6  | Pas de champ conditions sur <code>menu</code>                                                 | Ajout de la colonne <code>conditions</code> <code>TEXT</code>                                                 |
| 7  | <code>commande</code> sans clé primaire numérique                                             | Ajout <code>commande_id</code> <code>AUTO_INCREMENT</code> + <code>numero_commande</code> <code>UNIQUE</code> |
| 8  | Champs de contact employé manquants (motif, mode de contact)                                  | Ajout <code>motif_modification</code> <code>TEXT</code> , <code>mode_contact</code> <code>ENUM</code>         |
| 9  | <code>description</code> de <code>menu</code> en <code>VARCHAR(50)</code> , insuffisant       | Passage à <code>TEXT</code>                                                                                   |
| 10 | Pas de lien entre <code>avis</code> et <code>commande</code>                                  | Ajout FK <code>commande_id</code> dans <code>avis</code> (avis lié à une commande terminée)                   |
| 11 | Table <code>contact</code> absente                                                            | Table <code>contact</code> pour persister les demandes du formulaire                                          |
| 12 | Pas de type sur <code>plat</code> (entrée / plat / dessert)                                   | Ajout <code>type_plat</code> <code>ENUM('entree', 'plat', 'dessert')</code>                                   |

Quelques corrections de types ont aussi été faites au passage (horaires en `TIME` plutôt que `VARCHAR`, prix en `DECIMAL` plutôt que `DOUBLE` pour éviter les erreurs d'arrondi monétaire).

#### 3.2 Schéma final (Mermaid)

Diagramme entité-association du schéma réellement implémenté ( `schema.sql` ). Les tables techniques `reset_password_request` et `messenger_messages` (internes à Symfony) ne sont pas représentées ici.



### 3.3 Choix notables du schéma

- **Soft delete plutôt que suppression.** Les comptes ( `utilisateur.actif` ) et les menus ( `menu.actif` ) ne sont pas supprimés mais désactivés. Supprimer un compte ou un menu casserait les relations historiques (commandes ou avis orphelins). Le soft delete préserve l'intégrité des données et reste réversible.
- **Numéro de commande lisible.** En plus de la PK technique auto-incrémentée, chaque commande porte un `numero_commande` au format `VG-2026-00001` , unique, qui sert de référence côté client et de clé naturelle côté MongoDB.
- **Cascade maîtrisée.** Les tables de liaison et les images se suppriment en cascade avec leur menu/plat parent. Les commandes et avis, eux, ne cascaden pas (données à conserver).

---

## 4. Diagramme de cas d'utilisation

Quatre acteurs, avec héritage des droits : l'utilisateur hérite du visiteur, l'administrateur hérite de l'employé.

Faute de notation UML native pour les cas d'utilisation dans l'outil de diagramme, la représentation prend la forme d'un graphe : les acteurs, les cas d'utilisation regroupés par espace (un code couleur par espace), et les relations (héritage entre acteurs, include, extend).

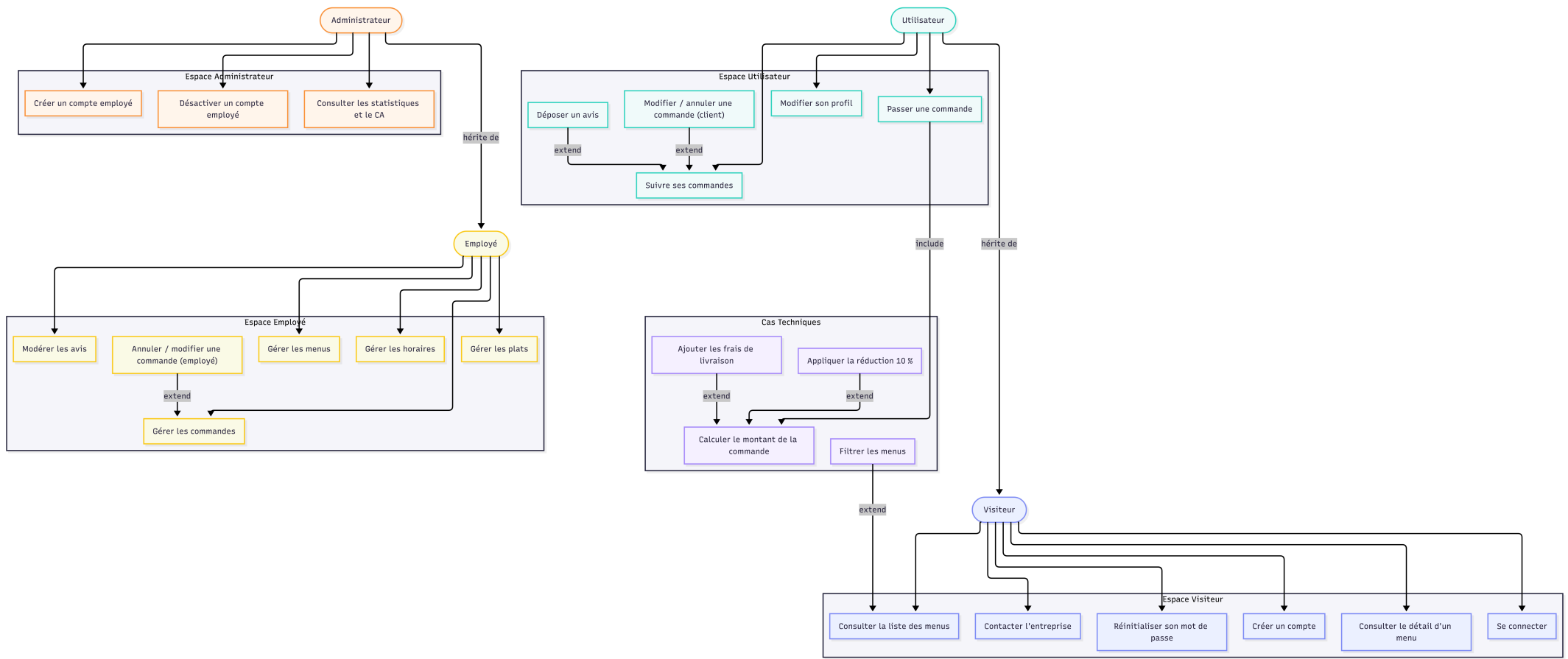
*Le diagramme de cas d'utilisation est présenté en page suivante, en orientation paysage pour rester lisible.*

Lecture du diagramme : chaque espace (visiteur, utilisateur, employé, administrateur, cas techniques) a sa couleur. Les flèches sans label relient un acteur à ses cas d'utilisation. Les flèches labellisées précisent le type de relation : "hérite de" (l'utilisateur reprend tous les cas du visiteur, l'administrateur tous ceux de l'employé), "include" (un cas toujours déclenché par un autre : passer une commande déclenche le calcul du montant), "extend" (un cas qui s'ajoute sous condition : réduction, frais de livraison hors Bordeaux, filtres, modification d'une commande). Les cas techniques ne sont reliés à aucun acteur : ils n'existent qu'au travers des relations include/extend.

Point de sécurité à souligner : aucun acteur ne peut créer un compte administrateur. Le compte admin est inséré uniquement par `seed.sql` (exigence du sujet). La création de compte employé par l'admin force le rôle "employé" en dur côté serveur.

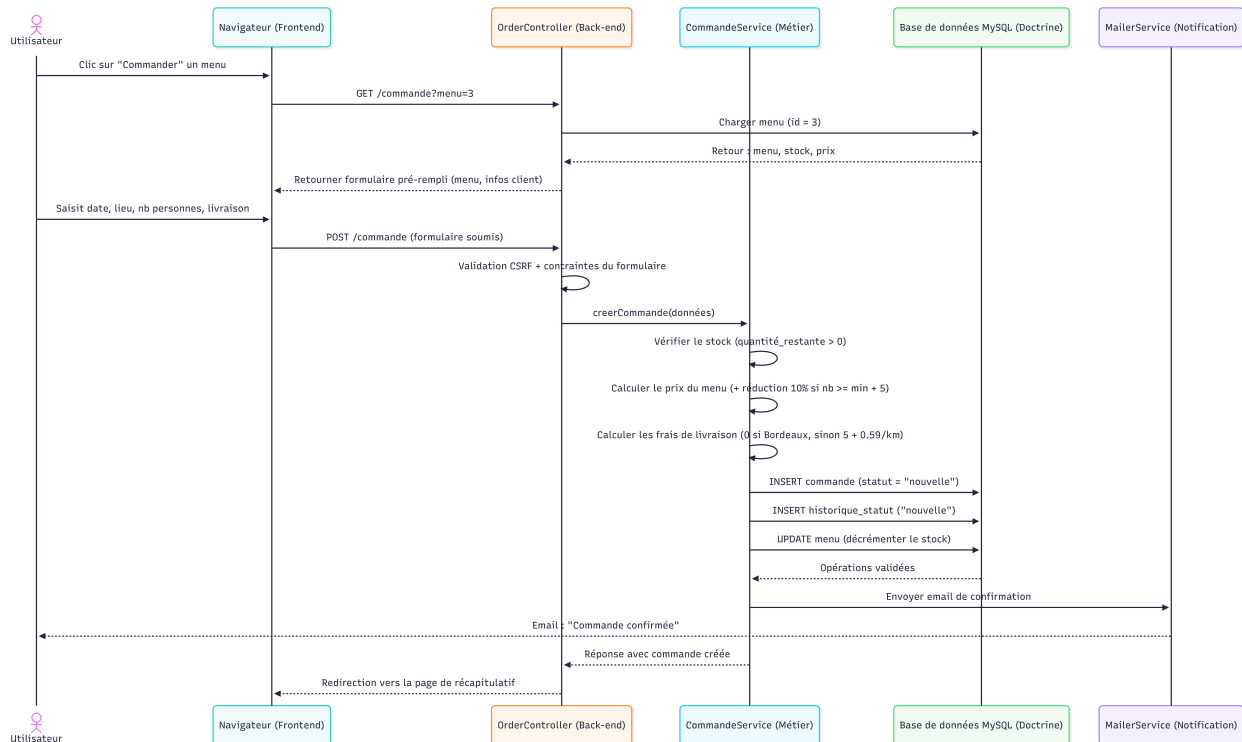


Diagramme de cas d'utilisation



## 5. Diagramme de séquence (commande d'un menu)

Le scénario décrit le parcours nominal d'un utilisateur connecté qui commande un menu depuis sa page détail. Il met en évidence la vérification du stock, le calcul du prix, l'écriture transactionnelle et l'envoi du mail de confirmation.



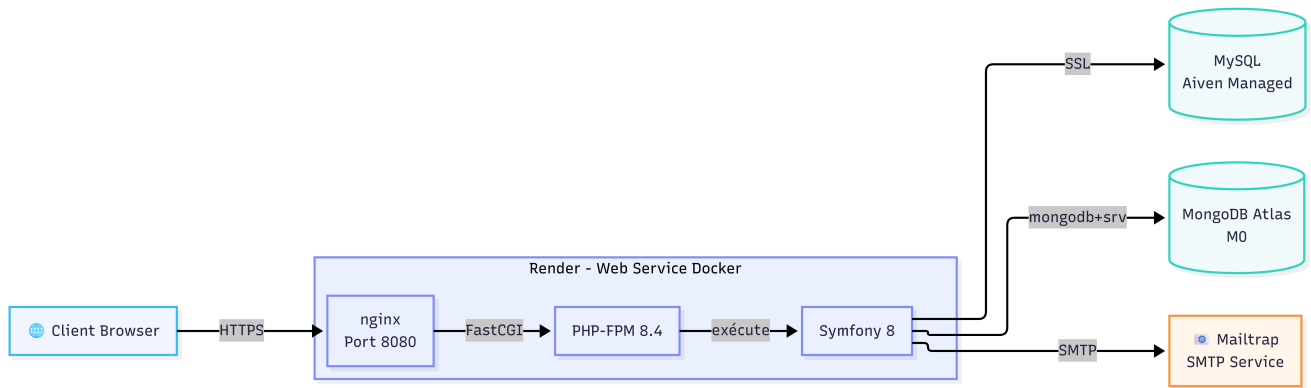
Règles métier appliquées dans `CommandeService` :

- **Prix du menu** : base = prix pour le nombre de personnes minimum. Au-delà, prix au prorata du nombre de personnes. Réduction de 10 % si le nombre de personnes est supérieur ou égal au minimum + 5.
- **Livraison** : gratuite dans Bordeaux, sinon 5 € + 0,59 € par kilomètre.
- **Stock** : vérifié avant validation, décrémenté après validation.
- **Traçabilité** : chaque commande crée une première entrée dans `historique_statut` , puis une entrée à chaque changement de statut (suivi complet exigé par l'énoncé).

## 6. Documentation du déploiement

### 6.1 Architecture de production

L'application est déployée sur Render sous forme de conteneur Docker. Les deux bases de données sont hébergées séparément, ce qui rend le conteneur applicatif sans état (stateless) et reproductible.



- **Application** : Render, web service Docker. À chaque push sur `main`, Render reconstruit l'image à partir du `Dockerfile` et redéploie. Le conteneur, basé sur l'image officielle `php:8.4-fpm`, fait tourner nginx + PHP-FPM pilotés par supervisor (un seul conteneur multi-process, compromis assumé pour le free tier). Note : le développement local se fait sous PHP 8.5.6, la production sur l'image stable 8.4, et l'application ne requiert que PHP 8.2 au minimum.
- **MySQL** : service managé Aiven, connexion SSL obligatoire (certificat CA embarqué dans l'image, configuré dans le bloc `when@prod` de `doctrine.yaml`).
- **MongoDB** : cluster Atlas M0, accessible via `mongodbsrv://` aussi bien en dev qu'en prod.
- **Mails** : Mailtrap (SMTP de test) capture les mails transactionnels.

## 6.2 Le fichier `render.yaml` et les secrets

Le service est décrit en infra as code dans `render.yaml` (runtime Docker, port 8080, health check). Les secrets de production y sont déclarés `sync: false` : leur nom est versionné mais leur valeur est saisie à la main dans le dashboard Render, jamais écrite dans le dépôt.

Variables à renseigner dans le dashboard Render :

- `APP_SECRET` (clé de chiffrement Symfony)
- `DATABASE_URL` (base Aiven, avec SSL)
- `MONGODB_URI`, `MONGODB_DB` (cluster Atlas)
- `MAILER_DSN` (Mailtrap)
- `DEFAULT_URI` (URL publique de l'application)

## 6.3 Étapes de mise en ligne

1. Charger une fois `schema.sql` puis `seed.sql` sur la base Aiven (connexion SSL requise) :  

```
bash mysql --host=<host-aiven> --port=<port> --user=avnadmin -p --ssl-mode=REQUIRED defaultdb < schema.sql
mysql --host=<host-aiven> --port=<port> --user=avnadmin -p --ssl-mode=REQUIRED defaultdb < seed.sql
```
2. Pousser le code sur GitHub (branche `main` : c'est la seule branche que Render déploie).
3. Sur Render : New -> Blueprint -> sélectionner le dépôt. Render lit `render.yaml`.
4. Renseigner les secrets `sync: false` dans le dashboard.
5. Lancer le déploiement et suivre les logs de build.
6. Une fois l'URL connue, corriger `DEFAULT_URI` avec l'URL réelle puis redéploier.
7. Vérifications en ligne : connexion admin, prise de commande, synchronisation Mongo, capture des mails dans Mailtrap.

## 6.4 Incidents rencontrés et corrections

- **Échec intermittent de `composer install` au build.** Composer 2 télécharge les dépendances en parallèle, ce que le serveur `code.load.github.com` rejetait parfois (erreurs HTTP 400). Correction : `ENV COMPOSER_MAX_PARALLEL_HTTP=1` dans le Dockerfile pour sérialiser les téléchargements. L'erreur venait de l'écosystème de dépendances, pas du code du projet.
- **Images absentes du site.** Le seed pointait vers `uploads/` (dossier gitignore, absent du repo) et le disque de Render est éphémère. Correction : j'ai intégré les images de démonstration comme assets statiques versionnés ( `public/images/` ), optimisées en WebP (63 Mo ramenés à 1,96 Mo), donc embarquées dans l'image Docker et toujours servies.

## 6.5 Limite assumée du free tier

Sur le free tier de Render, la machine se met en veille après 15 minutes d'inactivité, avec un cold start d'environ 50 secondes au réveil. Acceptable pour une démonstration. En production réelle, un plan payant ou un service de keep-alive supprimerait ce délai.

---

## Annexe : mécanismes de sécurité (résumé)

---

La sécurité est détaillée dans la copie à rendre (Partie 2.3). En résumé, le projet applique :

- **Authentification** : mots de passe hachés (algorithme automatique Symfony), politique de mot de passe forte (10 caractères, majuscule, minuscule, chiffre, caractère spécial), réinitialisation par lien signé.
- **Autorisation** : rôles hiérarchisés, `access_control` par espace, vérification de propriété (ownership) sur les commandes. Impossible de créer un admin depuis l'application.
- **Protections transversales** : `login_throttling` (5 tentatives/minute, anti brute force), en-têtes de sécurité via un EventSubscriber (X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS en HTTPS), cookie de session durci (Secure, HttpOnly, SameSite=lax).
- **Contre les injections** : requêtes paramétrées via Doctrine, échappement Twig automatique (XSS), jetons CSRF sur les formulaires.
- **Dépendances** : `composer audit` intégré à la démarche, montée en Symfony 8.0.13 pour corriger une CVE HIGH (CVE-2026-48489) et 8 autres advisories.
- **Secrets** : séparation stricte dev / prod, aucun credential versionné (Twelve-Factor App).